ÉGLISE GLORIEUSE DU CHRIST

Rapport complet de l'application EGC

État fonctionnel, statistiques, bugs, sécurité et capacité

Établi le 1er novembre 2026

> **Périmètre :** frontend React/Vite, application Android/Capacitor, API Express, base SQLite, intégration YouTube, déploiement Railway et vérifications effectuées sur le domaine public.

Ce document décrit explicitement ce qui fonctionne, ce qui est volontairement désactivé, comment les données circulent, quels bugs ont été rencontrés, quelles corrections ont été déployées et quelles limites restent à traiter.

Les statistiques de code proviennent du dépôt analysé. Les chiffres de charge de production ne sont pas inventés : aucun test de 5 000 utilisateurs simultanés n'a été exécuté sur Railway.

1. Résumé exécutif

L'application EGC est une application web responsive et Android destinée à présenter l'église, ses événements, la Bible, les prédications YouTube, les invitations et l'administration des inscriptions et annonces.

> **État global : fonctionnel pour le périmètre actuellement activé**, avec une réserve importante sur la montée en charge : le backend utilise encore SQLite synchrone et une seule instance est recommandée.

- **Frontend public Railway accessible :** `https://egliseglorieuseduchrist.up.railway.app`.
- **API de production accessible :** `https://egc-front-production.up.railway.app`.
- **Tests backend :** 6 tests réussis sur 6, dont 2 000 inscriptions simulées.
- **Build frontend :** Vite réussi.
- **Sécurité de base :** Helmet, CORS allowlist, JWT, bcrypt, Zod, requêtes paramétrées et rate limiting.
- **Limite :** PostgreSQL et Redis ne sont pas encore provisionnés ; le support garanti de 5 000 utilisateurs simultanés n'est donc pas démontré.

2. Statistiques détaillées du projet

37 494	1 060	26	17
lignes frontend source	lignes backend source	déclarations de routes React	routes API Express

Élément	Valeur constatée	Interprétation
Framework frontend	React 19 + Vite 7	Interface en composants, bundle optimisé par Vite, chargement différé de plusieurs écrans.
Navigation	React Router, 26 déclarations Route	Chaque écran est accessible par une URL interne, avec retour et navigation principale.
Backend	Express 5 + Node 22	API HTTP JSON, authentification administrateur et règles de validation serveur.
Base	SQLite, journal WAL, busy timeout 5 s	Bonne solution pour une instance et un trafic modéré ; limite la montée en charge horizontale.
Dépendances frontend directes	22 environ	React, Capacitor, cartes, icônes, formulaires, export XLSX et état local.
Dépendances backend directes	7	Express, bcrypt, CORS, dotenv, rate limit, Helmet, JWT et Zod.
Build frontend observé	Réussi, avertissement de gros chunks	Deux bundles Bible dépassent 4,9 Mo et 6,3 Mo avant gzip : amélioration possible par découpage.

3. Architecture et circulation des données

1. Le navigateur charge le bundle frontend depuis le service Railway frontend.
2. `frontend/src/services/apiConfig.js` pointe la production vers le service API dédié.
3. Les écrans publics lisent les annonces, vidéos et statut Live via des requêtes HTTPS.
4. Les annonces et les prédications sont conservées en cache local pour afficher un contenu de secours.

5. Les formulaires envoient du JSON au backend avec validation côté client et côté serveur.
6. Les opérations d'administration nécessitent un JWT de 20 minutes dans l'en-tête Authorization.
7. Le backend écrit les inscriptions, demandes de don et annonces dans SQLite.
8. YouTube fournit le flux RSS des vidéos du dimanche et la page du direct.

Service	Adresse / configuration	Rôle
Frontend	`egliseglorieuseduchrist.up.railway.app`	Application React publique, servie par Vite preview sur le port Railway.
API	`egc-front-production.up.railway.app`	Endpoints JSON, santé, annonces, vidéos, Live, inscriptions et administration.
Android	Capacitor	Bundle local Android ; les données distantes utilisent l'URL API de production.
YouTube	Chaîne EGC TV	Direct et quatre dernières vidéos dont le titre contient « dimanche ».

4. Fonctionnalités opérationnelles

Rubrique	Route	Fonctionnement détaillé	État
Accueil	`/`	Logo, présentation, carrousel d'actualités, verset du jour, raccourcis et vie de l'église.	**OK**
Événements	`/events`	Liste et détail des annonces ; les données viennent de l'API avec contenu local de secours.	**OK**
Direct	`/live`	Interroge l'API Live, affiche l'état du direct, le lecteur YouTube et quatre prédications du dimanche.	**OK avec dépendance YouTube**
Bible	`/bible`, `/LectureBible`	Lecture d'une Bible embarquée ; le contenu principal ne dépend pas de l'API.	**OK**
À propos	`/about`	Informations d'identité et de présentation de l'église.	**OK**
Invitation	`/invitation`	Message WhatsApp et lien public EGC ; téléchargement Android dans la copie frontend déployée.	**OK**
Localisation	`/localisation`	Carte de localisation de l'église.	**OK avec dépendance carte/réseau**
Administration	`/admin`	Connexion JWT, listes paginées, filtrage, annonces, uploads et exports administratifs.	**Protégé et testé**
Profil	`/profile`	Écran profil de l'application.	**Accessible**
Plans de lecture	`/PlanDeLecture`, `/plan-lecture-personnalise`	Plans bibliques locaux et personnalisation.	**Accessible**

Historique, rappels, encouragement	Routes dédiées	Modules locaux pour lecture, rappels et contenus d'encouragement.	**Accessibles**

5. Rubriques volontairement désactivées

Les composants d'origine sont conservés dans le code, mais les routes affichent temporairement l'écran « Bientôt disponible ». Sur l'accueil, les cinq rubriques sont cachées dans une section repliée « Nos futures versions » et apparaissent uniquement après clic.

Rubrique	Route	Comportement actuel	Code d'origine
Offrandes et dîmes	`/give`	Message « Bientôt disponible ».	Conservé et commenté dans App.jsx.
Nos formations	`/formations`	Message « Bientôt disponible ».	Formations.jsx conservé ; le fichier tagué contient 336 lignes.
Demande de prière	`/prayer`	Message « Bientôt disponible ».	Composant et ancienne route conservés.
Bibliothèques	`/testimony`	Message « Bientôt disponible ».	Composant de témoignage conservé.
Inscription	`/inscription`	Message « Bientôt disponible ».	Formulaire d'origine conservé.

6. Live et vidéos YouTube

- Le backend vérifie la page Live de la chaîne EGC TV.
- Lorsqu'un direct est détecté, l'API renvoie son identifiant et l'interface affiche le lecteur.
- Lorsque le direct est inactif, l'écran indique « Aucun direct n'est en cours » et affiche les prédications.
- Le flux RSS est filtré par le mot « dimanche » dans le titre.
- Les quatre vidéos distinctes les plus récentes sont retournées et mises en cache cinq minutes côté API.
- La vidéo du 1er novembre 2026 a été vérifiée comme valide avec l'identifiant YouTube `UqNR2ZX3dXE`.
- Une erreur YouTube 153 a été rencontrée dans un lecteur intégré ; la politique de référent a été corrigée vers `strict-origin-when-cross-origin`.

> **Limite :** la lecture dépend des règles YouTube, de l'autorisation d'intégration, de la disponibilité de la vidéo et de la connexion de l'utilisateur.

7. API et données

Famille	Endpoints principaux	Règles
Santé	`GET /`, `GET /api/health`	Réponse JSON de disponibilité.
Lecture publique	`GET /api/annonces`, `GET /api/videos`, `GET /api/live`	Pas d'authentification ; caches et repli explicites.
Écriture publique	`POST /api/inscription`, `POST /api/formations`, `POST /api/pay`	Validation Zod, rate limit, idempotence ; le paiement réel MTN n'est pas intégré.

Administration	Login, annonces, uploads, inscriptions, dons	JWT, réponses non mises en cache, pagination et contrôle de rôle.

8. Sécurité actuelle

- Helmet ajoute des en-têtes HTTP de protection.
- CORS limite les origines autorisées à une allowlist.
- Les secrets et mots de passe doivent venir des variables Railway ; le dépôt ne doit pas contenir de secrets.
- Les mots de passe sont hachés avec bcrypt.
- Les JWT sont vérifiés avec l'algorithme HS256 et expirent après 20 minutes.
- Zod limite les types, tailles, formats et valeurs des entrées.
- Les requêtes SQL utilisent des paramètres plutôt que de concaténer les entrées.
- Les limites de requêtes publiques et de connexion réduisent les abus simples.
- Les délais HTTP serveur sont bornés et les annonces publiques ont un cache court.
- Les uploads d'administration sont réservés au JWT et limités en taille et type.

> **Risque restant :** le rate limiting est local à l'instance. Avec plusieurs instances, il faut un store Redis partagé. SQLite synchrone doit être remplacé par PostgreSQL pour une montée en charge horizontale.

9. Bugs rencontrés et corrections

Bug	Cause	Correction	État
403 Vite sur le domaine public	Le domaine Railway n'était pas autorisé par le serveur preview.	Ajout de `preview.host` et `preview.allowedHosts`.	**Corrigé**
API appelée sur le mauvais domaine	Le frontend public demandait `/api/videos` au service frontend.	URL de production changée vers `egc-front-production.up.railway.app`.	**Corrigé**
Erreur YouTube 153	Référent refusé par le lecteur intégré.	Politique de référent corrigée.	**Corrigé côté application**
Anciennes vidéos vendredi/samedi	Expérimentations de filtrage et cache.	La logique Live a été restaurée à la version Railway demandée, puis le RSS filtre « dimanche ».	**Corrigé selon périmètre**
Futures rubriques trop visibles	La grille était rendue immédiatement.	Section repliable avec `aria-expanded` et ouverture au clic.	**Corrigé**
Footer invisible sur grand écran	Conteneur limité à la hauteur de la fenêtre et barre fixe recouvrant le bas.	Espace inférieur de défilement ajouté en desktop.	**Corrigé**
Liens sociaux inactifs	Les href étaient `#`.	Facebook, TikTok et YouTube EGC TV reliés aux liens fournis.	**Corrigé**
Migration ancienne base cassée après index	Index créés avant l'ajout de colonnes historiques.	Index déplacés après les migrations de colonnes.	**Corrigé et testé**

10. Tests et validations

Validation	Résultat	Détail
Tests backend	**6/6 réussis**	Idempotence, validation, sécurité, migration, vidéos, pagination et charge simulée.
Charge locale	**2 000 inscriptions simulées**	Test automatisé réussi en environ 17 secondes sur l'environnement local.
Build frontend	**Réussi**	Vite 7.3.7 produit le bundle web.
API Railway	**HTTP 200**	`/api/health` répond `{"status":"ok"}`.
Test public frontend	**Réussi**	Accueil, Live, Invitation et section repliable vérifiés dans le navigateur.
Charge réelle 5 000 utilisateurs	**Non mesurée**	Le test local ne constitue pas une garantie de capacité Railway.

11. Limites et bugs encore ouverts

1. **Scalabilité :** SQLite synchrone peut bloquer la boucle Node lors d'écritures concurrentes.
2. **Rate limiting distribué :** les compteurs ne sont pas partagés entre plusieurs instances.
3. **Pagination :** l'administration utilise encore OFFSET, moins efficace sur de très grandes pages.
4. **Chunks frontend :** les données bibliques produisent de gros bundles ; le premier chargement peut être lent sur mobile.
5. **Paiement :** `/api/pay` enregistre une intention ; il ne déclenche pas encore de paiement MTN réel ni de webhook signé.
6. **Live :** si la page reste ouverte avant le début d'un direct, un rechargement est nécessaire pour obtenir le nouveau statut avec la logique actuelle.
7. **Observabilité :** aucun tableau de bord p95/p99, tracing ou test de charge Railway automatisé n'est fourni dans le dépôt.

12. Plan recommandé pour dépasser 5 000 utilisateurs

Priorité	Action	Résultat attendu
1	Provisionner PostgreSQL Railway et migrer les tables SQLite avec scripts de migration et sauvegarde.	Écritures concurrentes non bloquantes, réplication et montée en charge.
2	Remplacer le rate limit mémoire par Redis partagé.	Limites cohérentes entre toutes les instances.
3	Ajouter un pool de connexions et des index validés par EXPLAIN.	Latence et débit plus prévisibles.
4	Mettre en place un test k6/autocannon avec 5 000 clients virtuels.	Mesure objective du débit, p95, p99 et taux d'erreur.
5	Découper la Bible en chargement différé et mettre les assets derrière CDN.	Premier affichage plus rapide, surtout sur mobile.
6	Ajouter logs structurés, alertes, sauvegardes chiffrées et suivi de santé.	Détection rapide des incidents et restauration contrôlée.

13. Conclusion

Le périmètre public actuellement activé est fonctionnel et les corrections demandées ont été déployées. Les cinq rubriques futures sont masquées puis accessibles au clic, sans suppression du code initial. Les liens sociaux et l'invitation pointent vers les adresses demandées. Le Live est relié à l'API et à YouTube.

La prochaine étape technique indispensable pour une promesse solide de plus de 5 000 utilisateurs est une migration PostgreSQL + Redis accompagnée d'un test de charge en environnement Railway. Jusqu'à cette étape, l'application doit être considérée comme robuste pour une instance et un trafic modéré, mais non comme une architecture multi-instance pleinement validée.

Sources : dépôt EGC analysé, tests backend, builds Vite, réponses API Railway et vérifications navigateur effectuées au 1er novembre 2026.